

Ministry of Science and Higher Education of the Republic of Kazakhstan NAO

L.N. Gumilyov Eurasian National University

Institute of Digital Sciences and Artificial Intelligence

Department of Computer and Software Engineering
(name of department)

EDUCATIONAL AND METHODOLOGICAL COMPLEX OF THE DISCIPLINE

COMP 22403 Programming Languages for Software and Device Development

(module code and name)

In the discipline OOPJ 3215 Object-oriented programming: Java

(full name of the discipline)

for students in the educational program

6B06104 – Computer Engineering and Software

(code and name of the OP)

**Non-Profit Joint-Stock Company «L.N. Gumilyov Eurasian National University»
«Institute of Digital Sciences and Artificial Intelligence»
Department Computer and software engineering education department**

Syllabus
**COMP 22403 Programming Languages for Software
Development and device**

on the discipline
OOPJ 3215 Object-Oriented Programming: Java for students of education program
6B06104 – Computer Engineering and Software

Астана
2026



This document, in accordance with paragraph 1 of Article 7 of the Law of the Republic of Kazakhstan dated January 7, 2003 «On electronic document and electronic digital signature» is equivalent to a document on paper

Document signed

№	Full name	Position
1	Demidchik Natalia	docent
2	Dyussekeyev Kanagat	Head of the кафедра
3	Davletova Ainash	Associate Professor (Associate Professor)
4	Orazaliyev Bakhytzhan	Director
5	Seilov Shakhmaran	Director of the Institute

Developer:

Демидчик Наталья Николаевна, docent

	JSC "Eurasian National L.N.Gumilyov University"	Working curriculum (Syllabus)	Edition: third
---	--	--------------------------------------	-----------------------

Working curriculum (Syllabus) for the discipline

OOPJ 3215 "Object-Oriented Programming: Java"

(code and name of the discipline)

developed on the basis of the educational program

6B06104 – Computer hardware and software

(code and name of the educational program)

Considered at the meeting of the “Computer and Software Engineering” department Protocol № 10 from 04.05.2026

Approved at the meeting of the Institute's Educational and Methodological Commission, Protocol № 9 from 14.05.2026

	NPJSC "Eurasian National L.N.Gumilyov University"	Working curriculum (Syllabus)	Edition: third
--	--	-------------------------------	----------------

EXPLANATORY NOTE

1. Brief description of the discipline

The course "Object-oriented programming: Java" is aimed at developing students' basic and applied knowledge in the field of object-oriented programming using the Java language, including decomposition of tasks into objects, encapsulation, construction of class hierarchies and polymorphism. Key elements of the OOP paradigm are studied, such as multiple inheritance, parametric polymorphism and exception handling. Particular attention is paid to the practical development of packages, interfaces, working with text and multithreaded programming.

The purpose of discipline	Learning outcomes (LO) for the educational program*	Expected learning outcomes (ELO) for the discipline
Students acquire basic theoretical knowledge and practical skills in software development using an object-oriented approach in the Java language	RT ₈ –Be able to make an optimal choice of data structures, be able to develop algorithms, and implement them in programming languages (PL), debug and analyze software source code and prepare reporting documentation	- Know the basics of developing and writing computer code to create and operate software using Java; - Be proficient in Java language tools that support the basic principles of an object-oriented approach (support for abstraction, encapsulation, hierarchy, modularity, typing, parallelism, and persistence in Java); - Know the basic concepts, models, methods and algorithms of the subject of object-oriented applications in Java; design patterns and architectural patterns for building information systems; - Be able to carry out development, and design the structure, content and writing of computer code necessary for the creation and operation of: system software, databases, object-oriented Java applications; - Possess practical skills in developing, modifying, and monitoring application software and services; - Be able to apply the acquired knowledge to develop the architecture of object-oriented applications using Java

2. Prerequisites

To master this discipline, knowledge, skills and abilities acquired through studying the following disciplines are required: Computer Science, Algorithms and Basics of Programming

Post-requisites The knowledge, skills and abilities acquired in the study of the discipline are necessary for studying the following disciplines: for the preparation of diploma theses

	NPJSC "Eurasian National L.N.Gumilyov University"	Working curriculum (Syllabus)	Edition: third
--	--	-------------------------------	----------------

3. Extract from the curriculum

Well 3

Semester 5

Number of credits 6

Types of classes	Total number of hours
Lectures	30
Practical lesson	-
Seminar session	-
Laboratory lesson	30
Studio lesson	-
SRO	120
Total	180

4. THEMATIC PLAN OF THE DISCIPLINE BY MODULES

(in academic hours)

Module No.	Module name
1	Introduction to Java Programming
2	Java Language Classes and Packages

Lectures				
Week #	Module No.	Lecture Topic	Quantity hours	Types and methods of teaching
1	1	Introduction to OOP. A brief history of the Java programming language. Object-oriented programming: basic concepts. Basics of classes and objects. Java machine versions and their evolution.	1	Introductory lecture with elements of problematic presentation (heuristic method (leading questions method))
2	1	Data types and operators. Conditional jump operator. Selection operator. Cycles. Arrays.	1	Informational lecture with demonstration elements (problem situation)
3	1	Classes and methods. Class variables, class instance variables, and constants. Access restrictions. Constructors. Method design.	1	Problematic lecture with elements of research method (research work)
4	1	Inheritance and polymorphism. Inheritance. Method overriding and polymorphism.	1	Visualization lecture with elements of analytical method (diagrams, charts, examples)
5	1	Inner classes. Nested classes. Anonymous classes.	1	Classification lecture with demonstration elements (comparative analysis, code demonstration)
6	1	Interfaces and annotations. Interfaces. Parameterization of interfaces. Annotations.	1	Conceptual lecture with elements of problematic presentation (conceptual map)
7	1	Functional interfaces. Predicate interface. Consumer interface. Comparator interface.	1	Practice-oriented demonstration lecture (optimization task)
8	2	Strings. String class. StringBuilder and StringBuffer. Regular expressions. Internationalization of the application.	1	Comprehensive informational lecture (performance comparison)

	NPJSC "Eurasian National L.N.Gumilyov University"	Working curriculum (Syllabus)	Edition: third
--	--	--------------------------------------	-----------------------

		Internationalization of numbers and dates. Date\Time API		
9	2	Exceptions and errors. Hierarchy of exceptions and errors. Methods of exception handling. Handling multiple exceptions. throw operator	1	Problem-analytical lecture (error analysis)
10	2	Input/output streams. Byte and character input/output streams. Reading from a stream. Scanner class.	1	Conceptual-applied lecture (schematization)
11	2	Collections and STREAM API. Collection hierarchy. ArrayList. LinkedList class and Queue interface. Deque interface and ArrayDeque class. STREAM API. Lambda expressions.	1	Review lecture with comparison elements (comparative analysis, practical examples)
12	2	Threads of execution. Thread class and Runnable interface. Thread life cycle. Thread management. and thread termination. Implementation of Runnable interface. Thread synchronization. synchronized operator	1	Problem lecture with elements of parallelism demonstration (problematization)
13	2	Java DataBase Connectivity. Drivers, connections and requests. Connection and request. Data Access Object	1	Technological lecture with elements of practical immersion (explanation of architectural pattern) DAO)
14	2	Network programs. Computer networks. Socket connections over TCP/IP. Multithreading. Servlets	1	Exploratory lecture with role-playing elements (servlet overview)
15	2	Java API for XML Processing. Tree model. Serialization and deserialization. DOM. Creating an XML document	1	Final practical lecture with demonstration elements (practical example of serialization/deserialization)
TOTAL			15	

Laboratory exercises				
Week #	Module No.	Lab Topics	Quantity hours	Types and methods of teaching
1	1	Introduction to OOP. Setting up the development environment. Console input/output. Tasks.	2	Workshop on mastering tools and basic syntax
2	1	Data types and operators. Conditional branch operator. Selection operator. Cycles. Arrays. Class development. Tasks	2	Training on basic structures and primary design
3	1	Classes and methods. Constructors. Method design. Tasks	2	Heuristic workshop on object design
4	1	Inheritance and polymorphism. Inheritance. Method overriding and polymorphism. Tasks	2	Problem-search work with object hierarchies
5	1	Inner classes. Inner classes. Nested classes. Anonymous classes. Tasks	2	Exploratory analysis of specific structures (coding by example)
6	1	Interfaces and annotations. Interfaces. Parameterization of interfaces. Annotations. Tasks	2	Programming by contract with elements of self-control
7	1	Functional interfaces. Predicate interface. Consumer interface. Comparator interface. Tasks	2	Applied training in functional style (programming with lambda expressions)

	NPJSC "Eurasian National L.N.Gumilyov University"	Working curriculum (Syllabus)	Edition: third
--	--	--------------------------------------	-----------------------

8	2	Strings. String class. StringBuilder and StringBuffer. Regular expressions. Internationalization of the application. Internationalization of numbers and dates. Date\Time API. Tasks	2	Information and practical master class (comparative test)
9	2	Exceptions and errors. Hierarchy of exceptions and errors. Methods of exception handling. throw operator. Tasks	2	Problem-diagnostic workshop (diagnostics and debugging)
10	2	I/O streams. Reading from a stream. Scanner class. Tasks	2	Applied experiment with data (experimental coding)
11	2	Collections and STREAM API. ArrayList. LinkedList class and Queue interface. Deque interface and ArrayDeque class. STREAM API. Tasks	2	Systematizing analysis of data structures and processing (problem solving)
12	2	Threads of execution. Thread class and Runnable interface. Thread management. Implementation of Runnable interface. synchronized operator. Tasks	2	Research workshop on parallel computing (debugging multi-threaded code)
13	2	Java DataBase Connectivity. Drivers, connections and queries. Connection and query. Data Access Object. Tasks	2	Integration workshop on working with databases (search task)
14	2	Network programs. Socket connections over TCP/IP. Multithreading. Tasks	2	Research and research project on network interaction (debugging network interactions)
15	2	Java API for XML Processing. Serialization and deserialization. DOM. Creating an XML document. Tasks	2	Final practical training on data exchange (parser development)
TOTAL			30	

SRO				
Week #	Module No.	SRO Topic Completion date	Quantity hours	Types and methods of teaching
1	1	Implementation of an arithmetic calculator. Testing	8	Project development with subsequent testing
2	1	Implementation of a graphical calculator. Interface	8	Exploratory research work on GUI design
3	1	Implementation of a graphical calculator. Logic programming and testing	8	Program and logic workshop (program development)
4	1	Implementation of a graphical application for knowledge testing. Classes.	8	Heuristic design of an object model (class diagram)
5	1	Implementation of a graphical application for knowledge testing. Graphical interface	8	Collaborative development/Peer review UI (study of patterns)
6	1	Implementation of a graphical application for knowledge testing: logic programming	8	Problem-analytical implementation (program development)
7	1	Implementation of a graphical application for knowledge testing: testing	8	Exploratory testing and regression analysis (report on testing results)
8	2	Implementation of a program for counting the frequency of occurrence of words in a file. Class development.	8	Conceptual data modeling (mini-project)
9	2	Implementation of a program for counting the frequency of occurrence of words in a file. Logic programming.	8	Algorithmic coding and optimization (program development, debugging)
10	2	Implementation of a program for counting	8	Checking correctness and

	NPJSC "Eurasian National L.N.Gumilyov University"	Working curriculum (Syllabus)	Edition: third
--	--	-------------------------------	----------------

		the frequency of occurrence of words in a file. Testing the application.		efficiency (testing report)
11	2	Implementation of a graphical application using a database (using a specific example). Creating classes	8	Application architecture design
12	2	Implementation of a graphical application using a database (using a specific example). Interface	8	Exploratory and creative creation of GUI (review of analogs)
13	2	Implementation of a graphical application using a database (using a specific example). Logic programming. Database programming.	8	Comprehensive development of system logic (mini-project)
14	2	Implementation of a graphical application using a database (using a specific example). Connecting a database.	8	Technical research and setup (troubleshooting, database connection)
15	2	Implementation of a graphical application using a database (using a specific example). Testing the application.	8	Final testing and validation (data validation, report)
TOTAL			120	

5 Brief organizational and methodological characteristics of the discipline

Types of control of educational achievements:

Rubicon 1 colloquium

Rubicon 2 colloquium

Final: a written exam is planned

Course Policies and Procedures

- do not be late for classes;
- do not miss classes without a good reason;
- promptly and regularly consolidate and develop the knowledge and skills acquired in lectures and laboratory classes through independent work using additional sources;
- fulfill all requirements of the academic schedule in a timely manner;
- creating an inclusive and equal learning environment where every student feels valued and respected. The learning process takes into account the individual characteristics of each student, including differences in abilities, learning styles, cultural and social background.

6 Educational and methodological support of the discipline

№ п/п	Author, title, year of publication	Information carrier	Available in stock (pcs.)		
			In the library	World Digi- tal Libraries	At the depart- ment
Main literature					
1	Walls, C. (2022). <i>Spring in Action</i> : [Учебное пособие] / C. Walls. — Shelter Island : Manning Publications, 2022. — 784 с.	study guide	-	-	1
2	Deitel, P. J., & Deitel, H. M. (2020). <i>Java How to Program, Early Objects</i> : [Учебное пособие] / P. J. Deitel, H. M. Deitel. — Boston : Pearson, 2020. — 1320 с.	study guide	-	-	1
3	Kryzhanovskaya, Yu. A. JAVA Basics: a tutorial	study guide	-	Лань : елец-	

	NPJSC "Eurasian National L.N.Gumilyov University"	Working curriculum (Syllabus)	Edition: third
--	--	-------------------------------	----------------

	/ Yu. A. Kryzhanovskaya, V. G. Lyalikova, M. M. Bezryadin. - Voronezh: VSU, 2020. - 96 p.			tronic library system. — URL: https://e.lanbook.com/book/433010 (дата обращения: 04.06.2025).	
4	Fedorichev, L. A. Implementation of Multi-threading in the Java Language: a textbook for universities / L. A. Fedorichev, O. V. Bukunova. - 2nd ed., reprinted. - St. Petersburg: Lan, 2025. - 72 p. - ISBN 978-5-507-52722-9.	study guide	-	Лань : electronic library system. — URL: https://e.lanbook.com/book/457502	
5	Lyalikova, V. G. Basic Java classes: a tutorial / V. G. Lyalikova, Yu. A. Kryzhanovskaya, M. M. Bezryadin. - Voronezh: VSU, 2020. - 80 p.	study guide	-	Лань : electronic library system. — URL: https://e.lanbook.com/book/433007 (дата обращения: 04.06.2025)	
Additional literature					
6	Sierra, K., & Bates, B. (2005). <i>Head First Java</i> : / K. Sierra, B. Bates. — Sebastopol : O'Reilly Media, 2005. — 688 c.	study guide	-	-	1
7	Bloch, J. (2018). <i>Effective Java</i> : [Учебное пособие] / J. Bloch. — Boston : Addison-Wesley Professional, 2018. — 416 c.	study guide	-	-	1
Electronic and Internet resources					
8	Joshua Bloch - Effective Java (3rd). 466 p. · 2019. — https://vk.com/wall-111905078_9612 (дата обращения 04.06.2025)				
9	Walls, C. (2022). <i>Spring in Action</i> : [Учебное пособие] / C. Walls. — Shelter Island : Manning Publications, 2005. — 784 c. - https://linux.ime.usp.br/~henrick/[ebook]20Manning%20-%20Spring%20in%20Action.pdf (дата обращения 04.06.2025)				
10	Introduction to Programming Using Java David. J. Eck/Department of Mathematics and Computer Science https://www.iitk.ac.in/esc101/share/downloads/javanotes5.pdf				
11	Mike Keith & Merrick Schincariol & Massimo Nardone. Pro JPA 2 in Java EE 8: An In-Depth Guide to Java Persistence APIs. 771 p. · 2018. - URL: https://www.pdfdrive.com/pro-jpa-2-in-java-ee-8-an-in-depth-guide-to-java-persistence-apis-e158453877.html (дата обращения 04.06.2025)				

7. System of assessment of the results of educational achievements of students

Students' knowledge, skills and abilities are assessed according to the following system

Letter Grading	Digital equivalent of points	% content nie	Traditional system assessment	Criteria for nomination
A	4.0	95-100	Great	Grade A is given if the student: a) has a complete and systematic understanding of OOP concepts (encapsulation, inheritance, polymorphism, abstraction) and Java syntax; understanding of design principles, working with exceptions, collections, in-

	NPJSC "Eurasian National L.N.Gumilyov University"	Working curriculum (Syllabus)	Edition: third
--	--	-------------------------------	----------------

				put/output streams, multithreading, JDBC and network interactions. b). has the ability to independently design, develop and test complex, high-quality object-oriented Java applications. Ability to write optimal, structured code that complies with standards, with effective exception handling and commenting. Effective use of development tools and a debugger. c) the student completed the SRO tasks in full, at a high level, demonstrating a creative approach, initiative and in-depth analysis of the decisions made.
A-	3.67	90-94		Grade A- is given if the student: a). demonstrates a complete and deep understanding of the main concepts of OOP and Java. There may be minor, insignificant shortcomings or inaccuracies in detailed explanations of some advanced topics (e.g., the specifics of garbage collector operation, thread subtleties). b) is able to design and implement complex applications in Java. The code is of high quality, structured, generally complies with standards, functional and reliable. Shortcomings in the code (e.g., minor style violations, rare non-optimal solutions) are corrected independently with minimal guidance. c) completed all SRO tasks in full and at a high level. Shortcomings in answers or solutions are usually corrected independently during the defense or discussion.
B+	3.33	85-89	Fine	The B+ grade is given when the student: a) demonstrates a solid knowledge of most of the basic concepts of OOP and Java. Confidently applies OOP principles, understands their role. There is an understanding of working with basic APIs (collections, exceptions, I/O). b). is able to design and develop applications of medium complexity. The code is functional, generally structured, but may have individual non-optimal solutions or deviations from standards. Exception handling is present, but may not always be exhaustive. c). completed the SRO tasks completely and correctly, but the completed tasks may not always be original or deeply optimized.
B	3.0	80-84		Grade B is given if the student: a). demon-

	NPJSC "Eurasian National L.N.Gumilyov University"	Working curriculum (Syllabus)	Edition: third
--	--	-------------------------------	----------------

				strates a good understanding of the basic concepts of OOP and Java, but there may be individual gaps or inaccuracies in understanding more complex or less frequently used elements (for example, some aspects of the Stream API, the specifics of network programming). b). is able to implement programs of average complexity. The code is working, but may contain non-optimal fragments, may not always comply with coding standards, and may require revision in terms of error handling. c). completed the SRO tasks, but the tasks may contain minor flaws that do not affect the overall functionality, or require minor revisions.
B-	2.67	75-79		Grade B- is assigned when the student: demonstrates sufficient understanding of the basic concepts of OOP and Java, but there may be difficulties or inaccuracies in explanation. Understanding of some topics may be superficial. b). can implement most tasks, but when designing or coding, non-optimal or incorrect solutions may arise. The code may be less readable, require significant revision in style and exception handling. c). the SRO tasks are completed, but may have noticeable shortcomings that require correction.
C+	2.33	70-74		Grade C+ is given when the student: a). demonstrates an understanding of the basic concepts of OOP and Java, but has difficulty applying or explaining more complex aspects. There may be errors in terminology. b). can complete simple tasks, but has significant difficulties in solving complex problems. The code is functional, but contains errors, non-optimal solutions, does not meet standards. Requires help with debugging. c). the SRO tasks are completed with significant shortcomings that require significant revision or explanation.
C	2.0	65-69	Satisfactorily	Grade C is given when the student: a). demonstrates understanding of only part of the basic concepts of OOP and Java. Has difficulty explaining relationships. b). is able to implement only the simplest software solutions. The code contains sig-

	NPJSC "Eurasian National L.N.Gumilyov University"	Working curriculum (Syllabus)	Edition: third
--	--	-------------------------------	----------------

				nificant errors, is not fully functional, does not meet the requirements. Needs constant assistance. c). SRO assignments are completed partially and with numerous errors.
C-	1.67	60-64		The grade C- is given when the student: a). demonstrates an extremely superficial understanding of most topics. Knowledge is fragmented. b). can solve only elementary problems, and the code will contain critical errors. Requires constant monitoring and assistance from the teacher. c). the SRO tasks are completed at a low level or not completed in full.
D+	1.33	55-59		The grade D+ is given when the student: a). has fragmentary and disparate knowledge on the topic of the question with significant errors in definitions. Does not understand the connection between a given concept, theory, phenomenon and other objects of the discipline. b). is not capable of independently solving most practical problems. The code demonstrates a lack of understanding of the basic principles. c). the performance of SRO tasks is extremely unsatisfactory, or most of the tasks are not completed.
D	1.0	50-54		The grade D is given when the student: a). demonstrates disjointed and fragmentary knowledge on the topic of the question, with significant errors in the definitions of the main terms, concepts and characteristics of facts/phenomena of OOP and Java. Understanding of the relationships between basic OOP concepts (classes, objects, inheritance) is minimal or absent. b). is able to implement only the simplest, elementary programming tasks, often with critical errors, inoperability or a fundamentally incorrect approach. c): the SRO tasks are completed unsatisfactorily, contain multiple gross errors, or a significant part of the tasks is not completed.
FX	0.5	25-49	Unsatisfactory	The "unsatisfactory" grade corresponds to the letter FX, F, which has a digital equivalent of 0 and a percentage content of 0-49.

	NPJSC "Eurasian National L.N.Gumilyov University"	Working curriculum (Syllabus)	Edition: third
--	--	--------------------------------------	-----------------------

F	0	0-24		This grade is given if the student a). reveals gaps in knowledge of the basic material provided by the program, has not mastered more than half of the discipline program. Made fundamental mistakes in the answers. b). is not able to complete most of the tasks provided by the program, or demonstrates a complete lack of understanding of the task. c). individual tasks are not completed, the basic literature is not worked through.
----------	----------	-------------	--	--

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

Lectures

Lecture 1: Introduction to Object-oriented Programming (OOP) and the Evolution of the Java Language

1. Introduction to Object-oriented Programming (OOP)

Object-oriented programming (OOP) is a programming paradigm based on the concept of "objects" that are instances of classes. These objects can contain data (fields, also called attributes or properties) and code (methods).

OOP provides a modular program structure, simplifying complexity management, code reuse, and support. Basic principles of OOP:

- **Encapsulation:** Combining data and methods that work with it in a single unit.
- **Abstraction:** hiding a complex implementation and showing only the important characteristics of an object.
- **Inheritance:** A mechanism that allows one class to inherit the properties and methods of another.
- **Polymorphism:** the ability of objects to respond differently to the same method calls.

2. A brief history of the Java programming language

Java was developed by James Gosling and his team at Sun Microsystems in the early 1990s. The language was originally called Oak, but in 1995 it was named Java. Its philosophy is "write once, run everywhere": compiled Java code can run on any platform that supports the Java Virtual Machine (JVM).

Key stages:

- **1995:** Java 1.0 was released. The basic structure of the language and JVM were introduced.
- **1998:** Java 2 (J2SE 1.2 — - added Swing and the collections framework.
- **2004:** Java 5-generalizations, annotations, enumerations, auto-unpacking.
- **2011-2017:** Java 7, 8 and 9-lambdas, Stream API, Modules.
- **Since 2018:** accelerated release cycle. Java 11 and later — performance improvements, new APIs, and LTS releases.

3. Basic concepts of OOP in Java

- **Class:** template for creating objects. Defines the object's fields and methods.
- **Object:** an instance of the class. Represents a real entity with a unique identity, state, and behavior.
- **Method:** A block of code that performs a specific task. Called by the object.
- **Constructor:** a special method for initializing objects.
- **Field/Property:** A variable that contains the object's data or state.

Example:

```
public class Car {
    String color;
    int speed;

    public Car(String color, int speed) {
        this.color = color;
        this.speed = speed;
    }

    void accelerate() {
        speed += 10;
    }
}
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

In this example, Car is a class. Each Car object will have a color and speed, and the accelerate method increases the speed.

4. The Java Virtual Machine (JVM) and its evolution

The JVM is a Java bytecode execution engine that provides platform independence. Over time, the JVM implementation has improved in performance, memory management, and support for modern architectures.

Significant JVM versions:

- **Java SE 8 (2014):** Stream API, lambdas, new Date / Time API.
- **Java SE 11 (2018):** first LTS after Java 8. Removal of deprecated APIs, new HTTP client.
- **Java SE 17 (2021):** Another LTS release that includes sealed classes, templates, and JEP enhancements.

Each new version of the JVM supports new language features, optimizes performance, and maintains backward compatibility.

Conclusion

Understanding the basics of OOP and the Java language is an important step towards developing reliable, modular, and maintainable software. Java's evolution from an embedded language to a leading platform for enterprise and mobile development underscores its versatility and relevance.

Lecture 2: Data types and operators. Conditional statements. The selection operator.

Cycles. Arrays

1. Data types in Java

Java provides two main types of data types: **primitive** and **reference**.

- **Primitive data types:**
 - byte, short, int, long-integer types
 - float, double — floating-point numbers
 - char — character type
 - boolean-boolean type (true/false)
- **Reference data types** are any class objects, arrays, and interfaces.

2. Operators in Java

Java supports many operators:

- Arithmetic operations (+, -, *, /, %)
- Comparison operators(==, !=, <, >, <=, >=)
- Boolean values (& & ,||, !)
- Assignments (=, +=, -=, *= etc.)
- Increment/decrement (++ ,--)

3. Conditional statements

- **The if-else** statement allows you to perform certain actions depending on the condition:

```
if (x > 0) {
    System.out.println ("Positive number");
} else {
    System.out.println ("Zero or negative number");
}
```

4. Switch selection statement

Used to select one of several code execution options:

```
int day = 3;
switch (day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println ("Tuesday"); break;
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

```
case 3: System.out.println("Environment"); break;
default: System.out.println("Another day");
}
```

5. Cycles

Loops allow you to perform actions repeatedly:

- **for**-loop with the following parameters:

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

- **while**-a loop with a precondition:

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}
```

- **do-while**-a loop with a postcondition:

```
int i = 0;
do {
    System.out.println(i);
    i++;
} while (i < 5);
```

6. Arrays

An array is a data structure that contains a fixed number of elements of the same type:

```
int[] numbers = new int[5];
numbers[0] = 10;
numbers[1] = 20;
```

You can also initialize an array when declaring it:

```
int[] numbers = {1, 2, 3, 4, 5};
```

Iterating over an array:

```
for (int num : numbers) {
    System.out.println(num);
}
```

Conclusion

Data types, operators, conditional constructs, and loops are the foundation of procedural and object-oriented programming in Java. The ability to work with arrays and manage the flow of program execution is a key skill for novice programmers.

Lecture 3: Classes and methods. Class variables, class instances, and constants. Access restriction. Ad constructors. Designing methods

In object-oriented programming, a class is a template that objects are created from. It defines the properties (variables) and behavior (methods) of objects.

- Class variables (static) belong to the class itself, not to the object.
- Instance variables-individual for each object.
- Constants are declared with the final keyword and cannot be changed after initialization.

Access modifiers:

- public — accessible to everyone
- private — accessible only within the class

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

- protected — accessible within the package and to subclasses

Constructors — special methods called when creating an object. They initialize the object's fields:

```
public class Person {
    String name;

    public Person(String n) {
        name = n;
    }
}
```

Designing methods involves defining the return type, method name, parameter list, and method body:

```
public int sum(int a, int b) {
    return a + b;
}
```

Lecture # 4

Inheritance and polymorphism. Inheritance. Method overriding and polymorphism

LECTURE PLAN:

1. The concept of inheritance
2. Redefining methods
3. Polymorphism

1. The concept of inheritance

Inheritance is one of the most important principles of object-oriented programming (OOP), which is that one class (called a subclass or derived class) can inherit the properties and methods of another class (base class or superclass). In the Java programming language, inheritance is implemented using the extends keyword. In this case, the derived class gets access to all public and protected fields and methods of the base class.

Example:

```
class Animal {
    void makeSound() {
        System.out.println("Some sound");
    }
}
```

```
class Dog extends Animal {
    void bark() {
        System.out.println("Bark");
    }
}
```

In this example класс, the Dog class inherits методthe makeSound() method from классthe Animal class. Thus, an object of the Dog class can use both its own methods and those of the parent class.

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

Inheritance allows you to:

- Reuse the base class code
- Organize the class hierarchy
- Simplify program support and expansion

Java only supports single class inheritance, but multiple inheritance is possible through interfaces.

2. Redefining methods

Method overriding is a mechanism by which a subclass provides its own implementation of a method that is already defined in the base class. Overriding allows you to change the behavior of a parent class method for a specific subclass.

Redefinition conditions:

- The method must have the same signature (name, parameters)
- The return type must be compatible with the base method
- A base class method must not be declared as final, private, or static
- We recommend using the `@Override` annotation in the method being overridden.

Example:

```
class Animal {
    void makeSound() {
        System.out.println("Some sound");
    }
}
```

```
class Cat extends Animal {
    @Override
    void makeSound() {
        System.out.println("Meow");
    }
}
```

Here метод, the `makeSound()` method is overridden in класс the Cat class. When calling the method via the Cat object, the new implementation will be used.

3. Polymorphism

Polymorphism is a property where a single interface can be used to represent different types of objects. There are two types of polymorphism in Java:

- Compilation (static): implemented through method overloading
- Executive (dynamic): implemented by overriding methods

Thanks to polymorphism, objects of different subclasses can be accessed through a reference to their common superclass. This allows for flexibility and extensibility of programs.

Example:

```
class Animal {
    void makeSound() {
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

```

        System.out.println("Some sound");
    }
}

```

```

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Woof");
    }
}

```

```

class Cat extends Animal {
    @Override
    void makeSound() {
        System.out.println("Meow");
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Animal a;
        a = new Dog();
        a.makeSound(); // Outputs "Woof"
        a = new Cat();
        a.makeSound(); // Outputs "Meow"
    }
}

```

Thus, the behavior of the makeSound() method depends on the specific object referenced by the variable a.

Advantages of polymorphism:

- Versatility and code reuse
- Flexibility of the program architecture
- Ability to implement the Open / Closed Principle

Questions:

1. What is inheritance and how is it implemented in Java?
2. What is the difference between method overriding and overloading?
3. How does polymorphism work in practice?

Recommended sources:

- Eckel B. Philosophy of Java
- Horstmann K. Java. Professional Library
- Oracle Java Documentation (<https://docs.oracle.com/en/java/>)

Lecture # 5

Internal classes. Inner classes. Nested classes. Anonymous classes

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	---	---	-------------------------

LECTURE PLAN:

1. Introduction to inner classes
2. Types of internal classes
3. Static nested classes
4. Anonymous classes

1. Introduction to inner classes

In many cases, when designing classes, you need one class inside another. Java provides the ability to create such classes, called inner classes. An internal class is defined inside the body of another class and can have access to all its fields, including private ones.

Internal classes allow you to:

- Logically group classes that are used only in one place
- Improve encapsulation
- Make your code more readable and organized

There are several types of internal classes:

- Non-static inner classes
- Static nested classes
- Local classes (defined in methods)
- Anonymous classes

2. Inner classes

An internal class is declared inside another class without the static keyword. It is bound to an object of an external class and has access to all its members, even if they are declared as private.

Example:

```
class Outer {
    private int data = 10;

    class Inner {
        void show() {
            System.out.println("Data is: " + data);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Outer outer = new Outer();
        Outer.Inner inner = outer.new Inner();
        inner.show();
    }
}
```

3. Nested classes (static nested classes)

Unlike internal classes, nested classes are declared with the static keyword. They don't have access to non-static members of the outer class directly.

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	---	---	-------------------------

Example:

```
class Outer {
    static int data = 30;

    static class Nested {
        void display() {
            System.out.println("Data is: " + data);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Outer.Nested nested = new Outer.Nested();
        nested.display();
    }
}
```

Advantages of nested classes:

- Simplification of the program structure
- Less memory load, as they do not require creating an instance of an external class

4. Anonymous classes

An anonymous class is a local class with no name, created and defined in a single expression. It is usually used to implement interfaces or abstract classes in place, without creating a separate named class.

Example:

```
abstract class Animal {
    abstract void makeSound();
}

public class Main {
    public static void main(String[] args) {
        Animal cat = new Animal() {
            @Override
            void makeSound() {
                System.out.println("Meow");
            }
        };
        cat.makeSound();
    }
}
```

Anonymous classes are used:

- When working with event handlers in the GUI
- If you need to quickly implement the interface
- When a single use of the class is required

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

Questions:

1. What is an internal class and how is it used in Java?
2. What is the difference between an internal class and a static nested class?
3. When are anonymous classes used and what are their advantages?

Recommended sources:

- Eckel B. Philosophy of Java
- Horstmann K. Java. Professional Library
- Oracle Java Documentation (<https://docs.oracle.com/en/java/>)

Lecture # 6

Interfaces and annotations. Interfaces. Parameterization of interfaces. Annotations

LECTURE PLAN:

1. The concept of interfaces in Java
2. Parameterization of interfaces (generalizations)
3. Annotations in Java
4. Applying Annotations

1. The concept of interfaces in Java

An interface in Java is an abstract type that defines a set of methods without an implementation. Interfaces are used to describe the behavior that classes can implement. Interfaces allow you to achieve complete abstraction and multiple implementations.

The interface is declared using the interface keyword. All interface methods are public and abstract by default (before Java 8).

Example:

```
interface Printable {
    void print();
}
```

```
class Document implements Printable {
    public void print() {
        System.out.println("Printing document");
    }
}
```

Since Java 8, interfaces can contain default methods and static methods:

```
interface Drawable {
    default void draw() {
        System.out.println("Drawing default");
    }
    static void info() {
        System.out.println("Drawable interface");
    }
}
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

Interfaces allow you to implement multiple behavior inheritance, since a class can implement multiple interfaces:

```
interface A {
    void methodA();
}
interface B {
    void methodB();
}
```

```
class C implements A, B {
    public void methodA() {
        System.out.println("Method A");
    }
    public void methodB() {
        System.out.println("Method B");
    }
}
```

2. Parameterization of interfaces

You can use generics to create interfaces with type parameters. This increases flexibility and type safety.

Example:

```
interface Container<T> {
    void add(T item);
    T get(int index);
}
```

```
class Box<T> implements Container<T> {
    private List<T> items = new ArrayList<>();

    public void add(T item) {
        items.add(item);
    }

    public T get(int index) {
        return items.get(index);
    }
}
```

This parameterization allows you to use the interface with any data type:

```
Container<String> stringBox = new Box<>();
stringBox.add("Hello");
System.out.println(stringBox.get(0));
```

3. Annotations in Java

An annotation is a special language construct designed to add meta-information to code.

Annotations do not directly affect program execution, but can be used by the compiler or runtime environment (for example, frameworks).

F ENU 703-07-21 Educational and methodological complex of the discipline. Seventh edition

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

Examples of built-in annotations:

- @Override-specifies that the method overrides the superclass method
- @Deprecated-marks a method or class as deprecated
- @SuppressWarnings-suppresses compiler warnings

Usage example:

```
class Animal {
    void speak() {
        System.out.println("Animal speaks");
    }
}
```

```
class Dog extends Animal {
    @Override
    void speak() {
        System.out.println("Dog barks");
    }
}
```

You can declare your own annotations using the @interface keyword:

```
@interface Info {
    String author();
    String date();
}
```

```
@Info(author = "Ivan", date = "2025-06-18")
class Demo {
    // ...
}
```

4. Applying Annotations

Annotations are actively used in the following areas:

- Spring, Hibernate, and JUnit frameworks
- Data validation (for example, @NotNull, @Size)
- REST API (@GET, @POST, etc.)
- Internal documentation and code generation

Annotations can be analyzed using the reflection mechanism in runtime.

Questions:

1. What is an interface in Java and what are its features?
2. What does parameterization of interfaces do?
3. How are annotations applied in practice?

Recommended sources:

- Eckel B. Philosophy of Java
- Horstmann K. Java. Professional Library
- Oracle Java Documentation (<https://docs.oracle.com/en/java/>)

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

Lecture # 7

Functional interfaces. The Predicate interface. The Consumer interface. Comparator Interface

LECTURE PLAN:

1. The concept of functional interfaces
2. The Predict interface
3. The Consumer Interface
4. Comparator Interface

1. The concept of functional interfaces

A functional interface in Java is an interface that contains exactly one abstract method. These interfaces have been used to implement lambda expressions and method references since Java 8. The functional interface declaration is accompanied by the `@FunctionalInterface` annotation, which signals the compiler about the purpose of the interface.

Example:

```
@FunctionalInterface
interface Operation {
    int execute(int a, int b);
}

public class Main {
    public static void main(String[] args) {
        Operation sum = (a, b) -> a + b;
        System.out.println(sum.execute(3, 4)); // 7
    }
}
```

Functional interfaces underlie the Stream API, the `forEach` method, and other Java functionality.

2. The Predict interface

`Predicate<T>` is a built-in functional interface from the `java.util.function` package that represents a logical function of a single argument of type `T`. It returns a boolean value.

Usage example:

```
import java.util.function.Predicate;

public class PredicateDemo {
    public static void main(String[] args) {
        Predicate<String> isLong = s -> s.length() > 5;
        System.out.println(isLong.test("Java")); // false
        System.out.println(isLong.test("Predicate")); // true
    }
}
```

Методы Interface methods:

- `boolean test(T t)`
- default `Predicate<T> and(Predicate<? super T> other` — - logical And

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

- default Predicate<T> or(Predicate<? super T> other — - logical OR
- default Predicate < T> negate—) - logical NOT

3. The Consumer Interface

Consumer<T> is a functional interface that represents an operation that accepts a single argument and returns no result. Used for performing actions on an object (for example, displaying it on the screen, logging it).

Example:

```
import java.util.function.Consumer;
```

```
public class ConsumerDemo {
    public static void main(String[] args) {
        Consumer<String> printer = s -> System.out.println("Hello, " + s);
        printer.accept("World"); // Hello, World
    }
}
```

Методы Interface methods:

- void accept(T t)
- default Consumer<T> andThen(Consumer<? super T> after) - allows you to perform actions sequentially

4. Comparator Interface

Comparator<T> is an interface designed to compare two objects. It is used in sorting methods (Collections.sort, Arrays.sort) and implements the compare(T o1, T o2) method.

With Java 8, Comparator is a functional interface.

Example:

```
import java.util.*;
```

```
public class ComparatorDemo {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("John", "Alice", "Bob");

        Collections.sort(names, (a, b) -> a.compareTo(b));
        System.out.println(names); // [Alice, Bob, John]
    }
}
```

Methods:

- int compare(T o1, T o2)
- static <T> Comparator<T> comparing(Function<? super T, ? extends U>)
- default Comparator<T> thenComparing(Comparator<? super T>)
- static <T> Comparator<T> reverseOrder()

Questions:

1. What is a functional interface and how is it related to lambda expressions?

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

2. How is the Predicate interface used and in what tasks is it useful?
3. What is the difference between the Consumer and Comparator interfaces?

Recommended sources:

- Eckel B. Philosophy of Java
- Horstmann K. Java. Professional Library
- Oracle Java Functional Interfaces
(<https://docs.oracle.com/javase/8/docs/api/java/util/function/package-summary.html>)

Lecture # 8

Strings. Класс The String class. StringBuilder и StringBuffer. Regular expressions. Internationalization of the app. Internationalization of numbers and dates. API Date/Time

ПЛАН ЛЕКЦИИ:

1. Working with strings: String, StringBuilder, StringBuffer
2. Regular expressions in Java
3. Internationalization: text, numbers, dates
4. Working with dates and times: java.time API

1. Working with strings: String, StringBuilder, StringBuffer

The String class in Java is an immutable sequence of characters. This means that any modification operations create a new object.

Example:

```
String s = "Hello";
s = s + "World"; // creating a new object
```

To optimize operations with strings, use:

- StringBuilder is a mutable class, not thread-safe, but fast.
- StringBuffer-mutable class, thread-safe (synchronized).

Пример Usage example:

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");
System.out.println(sb); // Hello World
```

2. Regular expressions in Java

Regular expressions (regex) allow you to check and process strings based on patterns. In Java, they are implemented in the java.util.regex class.Pattern and Matcher.

Example:

```
import java.util.regex.*;
```

```
public class RegexExample {
    public static void main(String[] args) {
        Pattern pattern = Pattern.compile("\\d{3}-\\d{2}-\\d{4}");
        Matcher matcher = pattern.matcher("123-45-6789");
        System.out.println(matcher.matches()); // true
    }
}
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

}

Frequently used templates:

- \d-digit
- \w — letter or number
- .- any character
- *, +, ? - quantifiers

3. Internationalization: text, numbers, dates

Internationalization (i18n) is the process of adapting software to different languages and regions without changing the source code. Java supports i18n via the `java.util.Locale`, `ResourceBundle`, `NumberFormat`, and `DateFormat` packages.

Example of number localization:

```
import java.text.*;
import java.util.*;
```

```
public class NumberFormatExample {
    public static void main(String[] args) {
        Locale fr = new Locale("fr", "FR");
        NumberFormat nf = NumberFormat.getCurrencyInstance(fr);
        System.out.println(nf.format(12345.67));
    }
}
```

Example локализации of date localization:

```
DateFormat df = DateFormat.getDateInstance(DateFormat.LONG, new Locale("de", "DE"));
System.out.println(df.format(new Date()));
```

4. Working with dates and times: `java.time` API

Java 8 introduces a new, more powerful and convenient API for working with dates and times — the `java.time` package. Main classes:

- `LocalDate` — date (year, month, day)
- `LocalTime` — time (hours, minutes, seconds)
- `LocalDateTime` — date and time
- `DateTimeFormatter`-formatting and parsing

Example:

```
import java.time.*;
import java.time.format.*;
```

```
public class DateTimeExample {
    public static void main(String[] args) {
        LocalDate date = LocalDate.now();
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd MMMM yyyy", new
Locale("ru"));
        System.out.println(date.format(formatter));
    }
}
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

Questions:

1. What is the difference between String, StringBuilder and StringBuffer?
2. How are regular expressions used in Java?
3. What is internationalization and how is it implemented in Java?
4. What features does the java.time API provide?

Recommended sources:

- Eckel B. Philosophy of Java
- Horstmann K. Java. Professional Library
- Oracle Java Documentation (<https://docs.oracle.com/en/java/>)

Lecture # 9

Exceptions and errors. Hierarchy of exceptions and errors. Methods for handling exceptions. Handling multiple exceptions. The throw statement

LECTURE PLAN:

1. The concept of exceptions and errors
2. Java exception Hierarchy
3. Exception handling
4. Handling multiple exceptions
5. Throw statement and creating custom exceptions

1. The concept of exceptions and errors

Exceptions and errors are objects that signal an abnormal situation during program execution. Exceptions can be caught and handled, but errors are usually not (for example, OutOfMemoryError).

Exceptions are divided into:

- Checked: IOException, SQLException, etc. — require processing or advertising
- Unchecked: ArithmeticException, NullPointerException-inherited from RuntimeException

2. Java exception Hierarchy

All exceptions are inherited from the Throwable class. It is divided into two subclasses:

- Exception — for normal exceptions (handled)
- Error — for critical JVM errors that should not be handled

Hierarchy:

- Throwable
 - Error
 - Exception
 - RuntimeException

Example:

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: Division by zero");
}
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

3. Exception handling

Java provides the try-catch-finally construct for exception handling.

Example:

```
try {
    int[] arr = new int[3];
    System.out.println(arr[5]);
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Index outside the range of the array");
} finally {
    System.out.println("БлокThe finally block выполнен is always executed");
}
```

Key points:

- try-block where an exception can occur
- catch-handling a specific type of exception
- finally — a block that is executed in any case

4. Handling multiple exceptions

You can process:

- Multiple catchblocks:

```
try {
    String s = null;
    s.length();
} catch (NullPointerException e) {
    System.out.println("NullPointerException");
} catch (Exception e) {
    System.out.println("General Exception");
}
```

- Multiple types in one catch:

```
try {
    int[] a = new int[5];
    a[5] = 10 / 0;
} catch (ArithmeticException | ArrayIndexOutOfBoundsException e) {
    System.out.println("Exception occurred:" + e);
}
```

5. Throw statement and creating custom exceptions

throw is used to explicitly throw an exception.

Example:

```
public class CustomExceptionDemo {
    public static void checkAge(int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Age must be >= 18");
        }
    }
}
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

```

    public static void main(String[] args) {
        checkAge(15);
    }
}

```

Custom exceptions are created by inheriting from Exception or RuntimeException:

```

class MyException extends Exception {
    public MyException(String message) {
        super(message);
    }
}

```

Questions:

1. What is the difference between checked and unchecked exceptions?
2. How is the exception hierarchy implemented in Java?
3. How do I use the throw statement to create custom exceptions?

Recommended sources:

- Eckel B. Philosophy of Java
- Horstmann K. Java. Professional Library
- Oracle Java Documentation (<https://docs.oracle.com/en/java/>)

Lecture # 10

Input/output streams. Byte and character input/output streams. Reading from the stream. The Scanner class

LECTURE PLAN:

1. Java I/O Streams
2. Byte streams
3. Character streams
4. Reading from the stream
5. The Scanner class and its features

1. Java I/O Streams

Streams in Java are used to transfer data between a source and a destination. All I/O classes are located in packages java.io and java. nio.

Streams are divided into:

- Байтовые (InputStream / OutputStream)
- Symbolic Links (Reader / Writer)

2. Byte streams

Byte streams are intended for reading and writing data in the form of bytes. They are used when working with binary files: images, audio, video, and so on.

Main classes:

- InputStream (an abstract class for reading bytes)
- FileInputStream

F ENU 703-07-21 Educational and methodological complex of the discipline. Seventh edition

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

- BufferedInputStream
- OutputStream (an abstract class for writing bytes)
- FileOutputStream
- BufferedOutputStream

Example of reading from a file:

```
import java.io.*;
```

```
public class ByteStreamExample {
    public static void main(String[] args) throws IOException {
        FileInputStream fis = new FileInputStream("file.txt");
        int b;
        while ((b = fis.read()) != -1) {
            System.out.print((char) b);
        }
        fis.close();
    }
}
```

3. Character streams

Character streams are used for working with text files where character encoding is important.

Main classes:

- Reader and Writer
- FileReader, BufferedReader
- FileWriter, BufferedWriter

Example:

```
import java.io.*;
```

```
public class CharStreamExample {
    public static void main(String[] args) throws IOException {
        FileReader fr = new FileReader("text.txt");
        int ch;
        while ((ch = fr.read()) != -1) {
            System.out.print((char) ch);
        }
        fr.close();
    }
}
```

4. Reading from the stream

For simplified reading, you can use buffering (BufferedReader) or the Scanner class, which allows line-by-line or post-word reading.

BufferedReader example:

```
BufferedReader reader = new BufferedReader(new FileReader("data.txt"));
String line;
while ((line = reader.readLine()) != null) {
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	---	---	-------------------------

```

    System.out.println(line);
}
reader.close();

```

5. The Scanner class and its features

The Scanner class from the java.util package provides a convenient way to read data from various sources: files, strings, and input streams.

Example of reading from the keyboard:

```

import java.util.*;

public class ScannerExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out. print ("Enter a name:");
        String name = scanner.nextLine();
        System. out. println ("Hello," + name);
        scanner.close();
    }
}

```

Example of reading from a file:

```

Scanner fileScanner = new Scanner(new File("data.txt"));
while (fileScanner.hasNextLine()) {
    System.out.println(fileScanner.nextLine());
}
fileScanner.close();

```

Methods:

- nextLine—) - string
- next () — word
- nextInt (), nextDouble() — numbers
- hasNext(), hasNextLine () - check for availability

Questions:

1. What is the difference between byte and character streams?
2. How does reading and writing work using the InputStream/Reader classes?
3. What features does the Scanner class provide?

Recommended sources:

- Eckel B. Philosophy of Java
- Horstmann K. Java. Professional Library
- Oracle Java I/O Documentation (<https://docs.oracle.com/en/java/>)

Lecture # 11

Collections and the STREAM API. Hierarchy of collections. ArrayList. The LinkedList class and the Queue interface. The Deque interface and the ArrayDeque class. STREAM API. Lambda expressions

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

LECTURE PLAN:

1. The hierarchy of collections in Java
2. ArrayList
3. LinkedList and Queue interface
4. Deque interface and ArrayDeque class
5. STREAM API Overview
6. Lambda expressions

1. The hierarchy of collections in Java

Collections in Java are collections of objects combined under the common Collection interface from the java.util package. Main interfaces:

- List — ordered collection (ArrayList, LinkedList)
- Set — set without duplicates (HashSet, TreeSet)
- Queue-структура "queue" structure (LinkedList, PriorityQueue)
- Deque-two-way queue (ArrayDeque)

2. ArrayList

The ArrayList class implements the List interface and is a dynamically modified array.

Peculiarities:

- Quick access by index
- Slow inserts and deletions in the middle of the list

Example:

```
import java.util.*;
```

```
public class ArrayListExample {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Java");
        list.add("Python");
        list.add("C++");
        System.out.println(list);
    }
}
```

3. LinkedList and интерфейс Queue interface

Класс The LinkedList class implements интерфейс the List and Queue interfaces. Suitable for frequent inserts and deletions.

Example of using it as a queue:

```
Queue<String> queue = new LinkedList<>();
queue.add("A");
queue.add("B");
System.out.println(queue.remove()); // A
```

Methods:

- add (), offer () — add
- remove (), poll () — delete

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

- peek—) - view the queue head

4. Deque interface and ArrayDeque class

Deque (two-way queue) - an interface that implements insert and delete operations on both sides.

ArrayDeque class-implementation of Deque on an array:

```
Deque<String> deque = new ArrayDeque<>();
deque.addFirst("A");
deque.addLast("B");
System.out.println(deque.removeLast()); // B
```

5. STREAM API Overview

STREAM API-a tool for functional processing of collections. Allows you to:

- filter, display, sort and collect data
- use chains of operations

Example:

```
List<String> names = Arrays.asList("Alice", "Bob", "Anna");
names.stream()
    .filter(name -> name.startsWith("A"))
    .forEach(System.out::println); // Alice, Anna
```

Basic methods:

- filter—) - filtering
- map—) - conversion
- collect—) - collecting the result
- sorted (), limit (), count() , etc..

6. Lambda expressions

A lambda expression is a compact way to define an anonymous function.

Syntax:

(parameters) -> expression

Example:

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
numbers.forEach(n -> System.out.println(n * n));
```

Questions:

1. What are the main interfaces and classes in the collection hierarchy?
2. What is the difference between ArrayList and LinkedList?
3. What is the STREAM API and why is it needed?
4. How do lambda expressions work in Java?

Recommended sources:

- Eckel B. Philosophy of Java
- Horstmann K. Java. Библиотека Professional Library
- Oracle Java Streams and Collections Documentation (<https://docs.oracle.com/en/java/>)

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

Lecture # 12

Execution threads. The Thread class and the Runnable interface. Life cycle of the thread.

Manage threads and terminate threads. Implementation of the Runnable interface.

Synchronization of streams. Synchronized statement

LECTURE PLAN:

1. The concept of multithreading in Java
2. Thread class and Runnable interface
3. Flow lifecycle
4. Flow management and completion
5. Thread synchronization and the synchronized statement

1. The concept of multithreading in Java

Multithreading is the ability to simultaneously execute multiple threads (flows of execution) within a single program. Java provides native support for multithreading through classes and interfaces in the java.lang and java.util.concurrent packages.

2. Thread class and Runnable interface

There are two ways to create a stream:

1. Inheritance from the Thread class
2. Implementation of the Runnable interface

Example 1: Inheritance from Thread

```
class MyThread extends Thread {
    public void run() {
        System.out.println ("Thread running");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

Example 2: Implementing Runnable

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println ("Stream via Runnable");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Thread t = new Thread(new MyRunnable());
        t.start();
    }
}
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

3. Flow lifecycle

Stream states:

- New-the thread was created, but not started
- Runnable-the thread is ready for execution
- Running-the thread is running
- Blocked/Waiting-the thread is waiting for a resource or signal
- Terminated — completed

Management methods:

- start () — starts the stream
- sleep — ms) - pause
- join () — waiting for the thread to complete
- interrupt () — interrupt

4. Flow management and completion

The join() method allows you to wait for another thread to finish:

```
Thread t1 = new Thread() -> {
    for (int i = 0; i < 5; i++) {
        System. out. println ("Stream 1");
    }
});
```

```
Thread t2 = new Thread() -> {
    try {
        t1.join();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    System. out. println ("Thread 2 после after thread 1 finishes");
});
```

```
t1.start();
t2.start();
```

5. Thread synchronization and the synchronized statement

Synchronization is a mechanism to avoid conflicts when accessing shared resources.

The synchronized statement is used:

- when declaring methods
- when blocking sections of code

Example:

```
class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }
}
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

```

    public int getCount() {
        return count;
    }
}

```

You can also sync a block:

```

synchronized (this) {
    // critical section
}

```

Questions:

1. What are the ways to create a stream in Java?
2. How does the thread lifecycle work?
3. What is synchronization and how is the synchronized operator used?

Recommended sources:

- Eckel B. Philosophy of Java
- Horstmann K. Java. Professional Library
- Oracle Java Concurrency Documentation (<https://docs.oracle.com/en/java/>)

Lecture # 13

Java DataBase Connectivity. Drivers, connections, and requests. Connection and request. Data Access Object

LECTURE PLAN:

1. General information about JDBC
2. Types of JDBC drivers
3. Establish a database connection
4. Executing SQL queries
5. The DAO (Data Access Object) template

1. General information about JDBC

JDBC (Java DataBase Connectivity) is a Java API for connecting and interacting with databases. Allows you to run SQL queries, get results, and manage transactions.

JDBC is supported by most popular DBMS systems: MySQL, PostgreSQL, Oracle, SQL Server, etc. Main components:

- DriverManager
- Connection
- Statement / PreparedStatement
- ResultSet

2. Types of JDBC drivers

There are four types of JDBC drivers:

1. JDBC-ODBC Bridge Driver
2. Native-API Driver
3. Network Protocol Driver
4. Thin Driver (pure Java)

F ENU 703-07-21 Educational and methodological complex of the discipline. Seventh edition

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

The most commonly used is Thin Driver (type 4), since it is fully implemented in Java and does not require additional libraries.

3. Establish a database connection

The connection is established via the DriverManager class, using the URL, username, and password.

Example of connecting to MySQL:

```
import java.sql.*;
```

```
public class DBConnect {
    public static void main(String[] args) throws Exception {
        String url = "jdbc:mysql://localhost:3306/testdb";
        String user = "root";
        String password = "password";

        Connection conn = DriverManager.getConnection(url, user, password);
        System.out. println ("Connection established");
        conn.close();
    }
}
```

4. Executing SQL queries

The Statement interface is used for executing queries:

```
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM users");
while (rs.next()) {
    System.out.println(rs.getString("name"));
}
```

For parameterized queries, use PreparedStatement:

```
String sql = "SELECT * FROM users WHERE id = ?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setInt(1, 1);
ResultSet rs = ps.executeQuery();
```

5. Шаблон The DAO (Data Access Object) template

A DAO is a design pattern that isolates the data access logic from the business logic of an application. The DAO provides:

- Clear separation of layers
- Ease of support
- Code reuse

Example:

```
public class User {
    private int id;
    private String name;
    // getters and setters
}
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

```

}

public interface UserDao {
    User getById(int id);
    void save(User user);
}

public class UserDaoImpl implements UserDao {
    private Connection conn;

    public UserDaoImpl(Connection conn) {
        this.conn = conn;
    }

    public User getById(int id) {
        // implementation
    }

    public void save(User user) {
        // implementation
    }
}

```

Questions:

1. What components does JDBC include?
2. What is the difference between the types of JDBC drivers?
3. How is data access implemented through the DAO?

Recommended sources:

- Eckel B. Philosophy of Java
- Horstmann K. Java. Professional Library
- Oracle JDBC Documentation (<https://docs.oracle.com/en/java/>)

Lecture # 14

Network programs. Computer networks. Socket connections over TCP / IP. Multithreading. Servlets

LECTURE PLAN:

1. Fundamentals of computer networks
2. Sockets in Java
3. TCP/IP socket connections
4. Multithreading in network applications
5. Introduction to Servlets

1. Fundamentals of computer networks

A computer network is a collection of devices connected for data exchange. Java supports the development of network programs via the API `java.net`.

Basic concepts:

F ENU 703-07-21 Educational and methodological complex of the discipline. Seventh edition

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

- IP address — a unique address on the network
- Port — an access point for data exchange
- Protocols-TCP (reliable) and UDP (unreliable)

2. Sockets in Java

A socket is a software interface for exchanging data between computers over a network. Java provides the Socket and ServerSocket classes.

Client side:

```
Socket socket = new Socket("localhost", 5000);
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
out.println ("Hello server!");
socket.close();
```

Server side:

```
ServerSocket serverSocket = new ServerSocket(5000);
Socket clientSocket = serverSocket.accept();
BufferedReader in = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
System.out.println ("Client:" + in.readLine ());
clientSocket.close();
serverSocket.close();
```

3. Сокетные соединения поTCP/IP socket connections

The TCP (Transmission Control Protocol) protocol provides reliable, connection-established data transfer between two points. It guarantees:

- Delivery of packages
- Orderliness
- Error checking

UDP (User Datagram Protocol) is faster, but does not guarantee delivery.

4. Multithreading in network applications

Multithreading allows you to process multiple clients simultaneously. Usually, the server creates a new thread for each client connection:

```
public class Server {
    public static void main(String[] args) throws IOException {
        ServerSocket serverSocket = new ServerSocket(5000);
        while (true) {
            Socket socket = serverSocket.accept();
            new Thread(() -> handleClient(socket)).start();
        }
    }
    private static void handleClient(Socket socket) {
        // processing the client
    }
}
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

5. Introduction to Servlets

Servlets are Java-based server components designed to handle HTTP requests. They work on the server side and are the foundation of Java EE.

Example of a servlet:

```
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.getWriter (). write ("Greetings from the servlet!");
    }
}
```

Servlets are deployed in containers such as Apache Tomcat or Jetty.

Questions:

1. What is a socket and how is it used in Java?
2. What are the differences between TCP and UDP?
3. How is multithreading implemented in network applications?
4. What are servlets and what is their role in web development?

Recommended sources:

- Eckel B. Philosophy of Java
- Horstmann K. Java. Professional Library
- Oracle Java Networking and Servlet Documentation (<https://docs.oracle.com/en/java/>)

Lecture # 15

Java API for XML Processing. Tree model. Serialization and deserialization of the. DOM. Creating an XML document

LECTURE PLAN:

1. Introduction to XML and its role in Java
2. Overview of the Java API for XML Processing (JAXP)
3. Tree Data Model (DOM)
4. XML serialization and deserialization
5. Creating and modifying XML documents using the DOM

1. Introduction to XML and its role in Java

XML (Extensible Markup Language) is a text-based format for representing structured data. XML is widely used for data exchange between systems. Java uses a set of APIs to work with XML, including DOM, SAX, and StAX.

2. Overview of the Java API for XML Processing (JAXP)

JAXP (Java API for XML Processing) provides standard interfaces for analyzing, generating, and transforming XML documents. It supports three approaches:

- DOM (Document Object Model)
- SAX (Simple API for XML)
- StAX (Streaming API for XML)

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

DOM is the most commonly used method for working with XML as a tree of objects.

3. Tree Data Model (DOM)

DOM is a model in which an XML document is represented as a tree of objects. Each element, attribute, and text node is represented as a Node object. The org.w3c.dom library is used for working with the DOM, and the DocumentBuilder class is used for building a tree.

Example:

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
Document doc = builder.parse(new File("data.xml"));
NodeList list = doc.getElementsByTagName("person");
for (int i = 0; i < list.getLength(); i++) {
    Element element = (Element) list.item(i);
    System.out.println(element.getElementsByTagName("name").item(0).getTextContent());
}
```

4. XML serialization and deserialization

Serialization is the process of saving objects as XML. Deserialization-restoring an object from an XML document.

For serialization, you can use TransformerFactory:

```
TransformerFactory tf = TransformerFactory.newInstance();
Transformer transformer = tf.newTransformer();
DOMSource source = new DOMSource(doc);
StreamResult result = new StreamResult(new File("output.xml"));
transformer.transform(source, result);
```

5. Creating and modifying XML documents using the DOM

Creating XML from scratch:

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
Document doc = builder.newDocument();
```

```
Element root = doc.createElement("people");
doc.appendChild(root);
```

```
Element person = doc.createElement("person");
root.appendChild(person);
```

```
Element name = doc.createElement("name");
name.appendChild(doc.createTextNode("John"));
person.appendChild(name);
```

Questions:

1. What approaches are available for processing XML in Java?
2. What is the DOM model?
3. How do I create and serialize an XML document in Java?

Recommended sources:

F ENU 703-07-21 Educational and methodological complex of the discipline. Seventh edition

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	---	---	-------------------------

- Eckel B. Philosophy of Java
- Horstmann K. Java. Professional Library
- Oracle JAXP and XML Documentation (<https://docs.oracle.com/en/java/>)

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	---	--	------------------

3. Plans for conducting laboratory classes and methodological recommendations for completing tasks

Methodological recommendations for performing laboratory work. The main emphasis in the methodology of conducting laboratory work should be made on developing and consolidating educational material in the process of performing tasks using computer technology in a computer classroom. Special attention should be paid to the application of elements of problem-based and contextual learning, as well as to students' advanced independent work. Current control of the assimilation of knowledge is carried out by preparing and submitting reports on the results of practical work, checking the completion of homework, surveys in practical classes.

1. Read the content of the theoretical part of the laboratory work.
2. Find additional information yourself.
3. Complete the lab tasks
4. Prepare a report for submission to the teacher.

Report design requirements The report must contain:

- name and goals of the work.
- results of the completed task.
- general conclusions made in the course of performing laboratory work.

Report protection is accompanied by a progress report

Laboratory work 1. Introduction to OOP. Install the development environment of the development environment. Console input/output. Tasks.

Option A

1. Greet any user when entering their name from the command line.
2. Display the command line arguments in the console window in reverse order.
3. Output the specified number of random numbers with and without transition move to a new line.
4. Enter the password from the command line and compare it with the sample string.
5. Enter integers as command-line arguments, calculate their sums and products. Output the result to the console.
6. Display the developer's last name, the date and time when the task was received, and date and time when the task was completed.

Option B

Enter n integers from the console. Output to the console:

1. Even and odd numbers.
2. The largest and smallest number.
3. Numbers that are divisible by 3 or 9.
4. Numbers that are divisible by 5 and 7.

Please answer the following questions:

1. What is meant when it says: Java is a programming language and a Java platform?
2. Decipher the abbreviations JVM, JDK, and JRE. Show where they are physically located and what they are.
3. JVM-JDK-JRE. Who includes whom and how they interact.

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	---	--	------------------

Laboratory work 2. Data types and operators. Conditional jump operator. The selection operator. Cycles. Arrays. Developing classes. Tasks

Task options

Option A

In the tasks below, enter the developer's last name, the date and time when the task was received, and the date and time when the task was completed. Add comments to programs in the form `/* comment */`. In number tasks, you can create an object as an array of characters. Enter n numbers from the console.

1. Find the shortest and longest number. Display the found numbers and their length.
2. Arrange and display the numbers in ascending (descending) order of values their lengths.
3. Output to the console those numbers whose length is less than (greater than) the average, as well as the length.
4. Find a number in which the number of distinct digits is minimal. If there are such there are several numbers, find the first of them

Option B

1. Display the multiplication table.
2. Output the array elements in reverse order.
3. Determine whether a certain value $of k$ belongs to the intervals (n, m) , $[n, m)$, $(n, m]$, $[n, m]$.
4. Display all numbers from 1 to 100 that are divisible by 3 without remainder.

Option C

Enter *the* n -dimension of the matrix $a[n][n]$ from the console. Set the values of matrix elements in the range of values from $-n$ to n using a random number generator.

1. Arrange the rows (columns) of the matrix in ascending order of values elements k of the k th column (row).
2. Perform a cyclic shift of the specified matrix by k positions to the right (left, up, down).
3. Find and output the largest number of ascending/descending elements matrices running in a row.
4. Find the sum of the matrix elements located between the first and second ones positive elements of each row

Please answer the following questions:

1. What Java primitive types exist, how to create variables of primitive types?
2. Explain the procedure by which variables of primitive types are passed to methods as parameters.
3. What is the size of primitive types? How the size of primitive types depends depends on the platform's bit depth?

Laboratory work 3. Classes and methods. Ad constructors. Designing methods. Tasks

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	---	--	------------------

Option A

Create classes whose specifications are listed below. Define constructors and methods **setType()**, **getType()**, and **toString()**. Define additional methods in the class that creates an array of objects. Set the criteria for selecting data and output this data to the console. Multiple constructors must be declared in each class that has information.

1. **Student:** id, Last Name, First name, Patronymic, Date of birth, Address, Phone number, Faculty, Course, Or Group. Create an array of objects. Output:

- a list of students of the given faculty;
- lists of students for each faculty and course;
- a list of students born after the given year;
- list of the study group.

2. **Customer:** id, Last Name, First Name, Patronymic, Address, Credit card number, Bank account number. Create an array of objects. Output:

- list of buyers in alphabetical order;
- a list of customers who have a credit card number in their possession. in the specified interval.

3. **Patient:** id, Last Name, First Name, Patronymic, Address, Phone number, Medical card number

maps, Diagnosis. Create an array of objects. Output:

- a list of patients with this diagnosis;
- a list of patients whose medical record number is in the specified interval.

4. **Abiturient:** id, Last Name, First Name, Patronymic, Address, Phone Number, Grades. Create an array of objects. Output:

- a list of applicants with unsatisfactory grades;
- a list of applicants who have a score higher than the specified one;
- select a given number n of applicants who have the highest qualification score. total points (also display the full list of applicants who have half-way amount).

Option B

Implement methods for adding, subtracting, multiplying, and dividing objects (for those classes whose objects can support arithmetic operations).

1. Define the class **Fraction (Rational Fraction)** as a pair of numbers m and n .

Declare and initialize an array of k fractions, and enter / output values for the array of fractions.

Create an array/list/set of objects and pass it to a method that modifies each element of the array with an even value.

index by adding the next element after it.

2. Define **the Complex class**. Create an array / list/set of dimension n from complex coordinates. Pass it to the method that executes addition/multiplication of its elements.

3. Define **the Quadratic Equation class**. Implement search methods roots, extremes, and descending/ascending intervals. To create array/list/set of objects and determine the largest and smallest roots by value.

4. Define the class **of a Polynomial** of degree n . Declare an array / list/set of m polynomials and determine the sum of the array's polynomials.

Option C

1. Define a class **of polynomials** with coefficients **of the Rational type**

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	---	--	------------------

Fraction. Declare an array / list/set of n polynomials and define the sum of the array's polynomials.

2. Define the class **of Straight** lines on a plane(in space), whose parameters are set using a **Rational Fraction**. Determine the intersection points of a straight line with the coordinate axes. Determine the intersection coordinates

two straight lines. Create an array/list/set of objects and define groups of parallel lines.

3. Define a class **of polynomials** with coefficients **of the Complex number type**.

Declare an array / list/set of m polynomials and define the sum array polynomials.

4. Define **the Fraction class** as a pair (m, n) with coefficients like

A complex number. Declare and initialize an array of k fractions, and enter / output values for the array of fractions. Create an array / list/set of objects and pass it to a method that modifies each element of the array with an even index by adding the next element after it.

Please answer the following questions:

1. Define such concepts as "class" and "object". Give examples of declaring a class and creating a class object. What specifiers

can I use it when declaring a class?

2. How do I determine which fields and methods need to be defined in the class?

Give an example. What specifiers can be used with fields, and what about the methods (and what do they mean)?

3. What is a constructor? How can I distinguish a constructor from any other method? How many constructors can a class have?

Laboratory work 4. Inheritance and polymorphism. Inheritance. Method overriding and polymorphism. Tasks

Option A

Create an application that meets the requirements specified in the task. Use inheritance only in tasks where it is logically justified. Argue that each method you create belongs to the class and correctly redefine **equals()**, **the equals()**, **hashCode()**, and **toString()** methods for each class **toString()**.

1. Create an object **of the Text class** using **the Sentence, and Word** classes.

Methods: add text, output text to the console, text title.

2. Create an object **of the Car class** using **the Wheel, and Engine** classes.

Methods: drive, refuel, change the wheel, display the car brand on the console.

3. Create an object **of the Airplane class** using **the Wing, Landing Gear, and Engine** classes.

Methods: fly, set a route, display a route to the console.

4. Create an object **of the State class** using **the Region, District, and District** classes.

City. Methods: display the capital, number of regions, area, and regional centers on the console.

Option B

Create a console application that meets the following requirements:

* Use OOP features: classes, inheritance, polymorphism, and encapsulation.

• Each class must have a meaningful name and informative content.

* Inheritance should only be applied when it makes sense.

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	---	--	------------------

- * Formatting conventions must be used when coding the java code convention code.
 - * Classes must be properly organized into packages.
 - * The console menu should be minimal.
 - You can use files to store initialization parameters.
1. **Flowergirl.** Define a hierarchy of colors. Create multiple color objects. Collect a bouquet (using accessories) and determine its cost. Sort the flowers in the bouquet based on their freshness level. To find a flower in the bouquet that corresponds to the specified range of stem lengths.
 2. **New Year's gift.** Define the hierarchy of candies and other sweets. Create multiple candy objects. Collect a child's gift with its weight determined. Sort the candies in the gift based on one of the following options: parameters. Find the candy in the gift that matches the specified range of sugar content.
 3. **Home electrical appliances.** Define the hierarchy of electrical appliances. Plug some of them into the wall outlet. Calculate the power consumption. Sort the appliances in the apartment based on power. Find a device in the apartment, corresponding to the specified range of parameters.
 4. **Chef.** Define the hierarchy of vegetables. Make a salad. Calculate the caloric content. Sort vegetables for salad based on one of the following options: parameters. Find vegetables in a salad that match the specified range caloric content.

Please answer the following questions:

1. Principles of OOP.
2. Rules for overriding the boolean **equals(Object o) method**.
3. Why redefine **hashCode()** the **hashCode()** and **equals()** methods **equals()** at the same time?
4. Write **the equals() method** for a class containing a single **String field**.

Laboratory work 5. Internal classes. Inner classes. Nested classes. Anonymous classes. Tasks

Option A

1. Create a **Notepad class** with an internal class or classes, using which can store multiple records for the same date.
2. Create a **Payment class** with an internal class, which objects can be used to create a purchase from several products.
3. Create an **Account class** with an internal class, which objects can be used to store information about all operations with the account (withdrawals, payments, receipts).
4. Create a class **Credit Book** with an internal class, which objects can be used to store information about sessions, credits, and exams.

Please answer the following questions:

1. What are internal, nested, and anonymous classes? How to determine classes of this type? How to create class objects of this type.
2. List the access capabilities to members of the external class that nested classes have?

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	---	--	------------------

Laboratory work 6. Interfaces and annotations. Interfaces. Parameterization of interfaces. Annotations. Tasks

Option A

Develop a process management project based on creating and implementing interfaces for the following subject areas:

- 1. Printing publications.** Opportunities: sign up for a contract; open\ edit/edit the publication; send it to print; cancel the publication; pay for the publication; resume/close the publication. Add specialized methods for a Book, Magazine, or Textbook.
- 2. Subscriber of the mobile operator.** Features: issue a contract; open an account and phone number; edit a subscriber's account; get all the information you need. up-to-date information on the subscriber's number; check the balance status and traffic balance; view details and information about payments; change the tariff plan, operator; top up the account; close the account and number. Add specialized methods for the Internet Account and corporate subscriber.
- 3. Candybars.** Features: release a batch of candies; get information on products of a certain manufacturer; add and edit product information; add new ingredients; create a new type of candy; remove a certain type of candy from production. Add specialized techniques for Candy, Chocolate, Lollipop, and Toffee.
- 4. The plant.** Features: set and edit plant information; get information about existing plants in the greenhouse/plants of a certain type/place of origin; purchase a new type of plant; make an order for a new plant. watering; set the temperature; set the lighting; dig out a certain area plant type (remove from the greenhouse). Add specialized methods for Bush plant, Flowering plant, Houseplant.

Please answer the following questions:

1. What is the interface? How do I define and implement an interface in a java program?
2. Is it possible to describe constructors and create objects in the interface?
3. Is it possible to create interface references, and if so, to which objects can they link to?

Laboratory work 7. Functional interfaces. The Predicate interface. The Consumer interface. The Comparator interface. Tasks

Option A

1. Use currying to implement the function of adding two numbers, the function of checking a string for a regular expression, the split function strings based on a regular expression.
2. Determine whether words are anagrams, i.e. whether it is possible from a single word make another one by rearranging the letters.
3. Write the User class with the following fields: id, name, age, and country. To create array of Users. Sort by country and age. Select all of them Users older than the specified age who have the first letter of their first name. it starts with the specified letter. Get the maximum and minimum values

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	---	--	------------------

element in the grouped result by age.

4. Write a functional interface with a method that accepts a number and returns a boolean value. Write an implementation of such an interface as a lambda expression,

Please answer the following questions:

1. Define a functional interface. **default & static** methods. Scope of application. How are they called?
2. What is a lambda expression? Its structure.
3. Short circuit. What variables does the lambda expression have access to?

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	---	--	------------------

Laboratory work 8. Strings. The String class. StringBuilder и StringBuffer. Regular expressions. Internationalization of the app. Internationalization of numbers and dates. API Date\Time. Tasks

Option A

1. In each word of the text, *replace the k-th* letter with the specified character. If *k* is longer than the word length, do not make any adjustments.
2. In the text, replace each letter with its ordinal number in the alphabet. By print text with two spaces between the letters in one line, in the next line at the bottom of each letter, print its number.
3. In the text after the letter P, if it is not the last one in the word, the letter A is mistakenly printed instead of O. Make corrections to the text.
4. *Insert the specified substring in the text after the kth character.*

Option B

Create a text processing program for a programming textbook using the following classes: *Symbol, Word, Sentence, Paragraph, Token, Listing, Punctuation Mark*, etc. In all text generation tasks, replace tabs and sequences of spaces with a single space. First, the text should be disassembled into its component parts, perform one of the tasks listed below, and display the result.

1. Find the largest number of text sentences that contain the same words.
2. Display all sentences of the given text in ascending order of the number of words in each of them.
3. Find a word in the first sentence that doesn't exist in any of them. all other offers.
4. In all interrogative sentences of the text, find and print words of the specified length without repetition

Option C

1. Check whether the string is a strong password. The password is considered strong, if its length is greater than or equal to 10 characters, it contains at least one digit, one uppercase letter, and one lowercase letter. Password it can only contain Latin letters and / or numbers, as well as the "_" symbol.
2. A text of *m* characters is encrypted according to the following rule:
— all text characters are recorded in a square dimension table *n* in order from left to right, from top to bottom — - the table rotates 90° clockwise; - the 1st row of the table changes places from the last, 2nd-from the penultimate, etc.; —the 1st column of the table changes places from the 2nd, 3rd - from the 4th, etc.; - encrypted text obtained by traversing the resulting tables spiral clockwise, starting from the upper-left corner. Encrypt the text according to the specified rule.
3. Exclude from the text a substring of the maximum length that starts and ends with the same character.
4. Delete the minimum number of sentences from the text so that any two remaining sentences had at least one word in common.

Please answer the following questions:

1. How to create an object **of the String class**, what class constructors exist **String**? What is a string literal? What does "simplified creation" mean? **объекта the String object**?"
2. How **is the String class implemented** and what fields are declared in it?

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	---	--	------------------

3. How does the `substring()` method of the `String` class work?

Laboratory work 9. Exceptions and errors. Hierarchy of exceptions and errors. Methods for handling exceptions. The `throw` operator. Tasks

Option A

The symbol file contains information about N floating-point numbers, specifying the locale of each number separately. Read information from the file. Check for correctness, i.e. whether the numbers are numbers. Convert to numeric values and calculate the sum and average of the calculated numbers.

Create your own exception class. Provide for handling exceptions that occur when there is a lack of memory, the file itself is missing at the specified address, the required entry in the file is missing or incorrect, an invalid number value (exceeding the maximum allowed values), etc.

Please answer the following questions:

1. What is an exceptional situation for the program? Which ones exist ways to handle errors in programs?
2. What is an exception for a Java program? What does "the program generated / threw an exception" mean? Give an example when exceptions occur generated by the VM (automatically) and when necessary generate them manually.
3. Create a hierarchy of exception classes that divides exceptions into checked and unchecked ones. What are the features of checked and unchecked exceptions?

Laboratory work 10. Input/output streams. Reading from the stream. The `Scanner` class. Tasks

Option A

In the following tasks, you need to enter a sequence of lines from the text stream and perform the specified actions. In this case, two options can be considered:

- each line consists of one word.
- each line consists of several words.

The names of the input and output files, as well as the absolute path to them, can be entered as command-line parameters or stored in a file.

1. Find and delete the specified substring in each line.
2. In each line of the poem, find and replace the specified substring with a substring of a different length.
3. Find words starting with a vowel in each line.
4. Find and display text words with the last letter of the same word matches the first letter of the next word.

Option B

Perform tasks from Option B of Chapter 4 by storing application objects in one or more files using the serialization mechanism. Objects can contain fields marked as **static** or **transient**. To change information and extract information in the file, create a special class connector with the necessary methods for performing these tasks.

F ENU 703-07-21 Educational and methodological complex of the discipline. Seventh edition

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	---	--	------------------

Option C

When performing the following tasks, create a new directory and file using the File class to display the results **File**.

1. Create and fill the file with random integers. Sort it file contents in ascending order.
2. Read the Java program text and replace all the words **public** in the class attributes and methods declaration with the word **private**.
3. Read the Java program text and write the characters of each line in reverse order to another file.
4. Read the Java program text and each word is longer than two characters replace all lowercase characters with uppercase ones.

Please answer the following questions:

1. What is a data stream? What data streams exist in Java?
Bring the hierarchy of I / O streams to Java.
2. What classes of byte I / O streams exist?
3. What classes of character I / O streams exist?

Laboratory work 11. Collections and STREAM API. ArrayList. The LinkedList class and the Queue interface. The Deque interface and the ArrayDeque class. STREAM API. Tasks

Option A

1. Enter lines from the file and add them to the list. Print the lines to the file in reverse order.
2. Enter a number and add its digits to the stack. Print a number with digits they go in reverse order.
3. Create a list of items in the directory and its subdirectories.
4. Add one author's poems to the list. Sort by increasing the length of strings.

Option B

1. There are N people in a circle, numbered from 1 to N. When maintaining an account for the circle is crossed out every second person until there is only one left.
Create two programs that simulate the process. One of the programs it should use **the ArrayList class**, and the second **one should use LinkedList**. Which of the two programs running faster? Why?
2. Given a list of integers and a certain number X. Without using auxiliary objects or sorting methods, and without changing the size of the list, rearrange the list elements so that first there are numbers not exceeding X, and then numbers greater than X.
3. Write a program that compresses English text.
Construct an optimal prefix code for each word in the text by the Huffman algorithm. Use **the PriorityQueue class**.
4. Implement **an undirected Graph class**, представляющий собой неориентированный graph. The class constructor passes the number of vertices in the graph.
Methods must support fast addition and removal of edges.

Please answer the following questions:

1. Name the main collection interfaces. What are collections?

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	---	--	------------------

2. What are the features of different types of collections? When and which collections should I use?

3. Compare **ArrayList** and **LinkedList**

Laboratory work 12. Execution threads. The Thread class and the Runnable interface. Flow control. Implementation of the Runnable interface. The synchronized statement. Tasks

Option A

Develop a multithreaded application. Use the features provided by the **java.util.concurrent** package. Don't use the word *synchronized*. All entities that want to access the resource must be threads.

1. **Port.** Ships enter the port to unload / load containers. Number containers that are currently in port and on board the ship should be non-negative and exceed the specified cargo capacity of the vessel and the port capacity. The port has several berths. One of them has
There can be only one ship berth. The ship can be loaded at the berth, unload or perform both actions.
2. **Small library.** Several books are available for reading. The same ones there are no books in the library. Some are given out on hand, some only to the reading room. The reader can pick up several books and bring them to the reading room.
3. **Parkinglot.** Several parking spaces are available. There can only be one vehicle in one place. If all seats are occupied, the car will not be able to
it will wait longer than a certain amount of time and then move to another parking lot.
4. **CallCenter.** The organization has several operators. The operator can serve only one client, the rest must wait for their own service.
queues. The client can hang up the phone and call back after a while.

Please answer the following questions:

1. What is the execution flow? Stream states.
2. How do I create a stream? What are the three ways to create a thread, start it, and abort (terminate, kill) it?
3. Life cycle of the thread.

Laboratory work 13. Java DataBase Connectivity. Drivers, connections, and requests. Connection and request. Data Access Object. Tasks

Option A

In each task, you must perform the following actions:

- organize the database connection in a separate class or method which the connection returns.
- * create a database. Convert tables to one of the normal forms.
- * create a class for executing queries to extract information from the database using compiled queries.
- * create a class for modifying information.

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	---	--	------------------

1. **File system.** The database stores information about the directory tree of the file system — directories, subdirectories, and files.

For directories, you must store:

- * the parent folder.
- name.

For files, you must store:

- * the parent folder.
- name.
- space used on Yandex. Disk.
- * Determine the full path of the specified file (directory).
- * Count the number of files in a given directory, including nested ones files and directories.
- * Calculate the disk space occupied by the contents of a given folder.
- * Find files in the database by the specified mask with the full path output.
- * Move files and subdirectories from one directory to another.
- * Delete files and directories in the specified directory.

2. **Video library.** The database stores information about your home video library: movies, actors and directors. For movies, you must store:

- name.
- * names of actors.
- the release date.
- the country where the movie was released.
- For actors and directors, you must store:
- FULL NAME.
- * date of birth.
- * Find all movies released in the current and last year.
- * Display information about the actors who starred in the given movie.
- * Display information about actors who have appeared in at least N films.
- * Display information about actors who directed at least one of the films.
- * Delete all movies that have a release date of more than the specified number of years back.

3. **Class schedule.** The database stores information about teachers and their classes. For items, you must store:

- name.
 - time of the event (day of the week);
 - * classrooms where classes are held.
- For teachers, you must store:
- FULL NAME.
 - * subjects that he leads;
 - * the number of pairs per week for each subject.
 - * the number of students engaged in each pair.
 - * Display information about teachers working on a given day of the week in a given classroom.
 - * Display information about teachers who do not teach classes on a given day of the week.
 - * Display the days of the week in which the specified number of classes is held.
 - * Display the days of the week when the specified number of audiences is occupied.
 - * Move the first classes of the specified days of the week to the last place.

Please answer the following questions:

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	---	--	------------------

1. What is JDBC? List the main classes and interfaces that are included in as part of JDBC, specify their purpose. What other technologies exist Java working with the database?
2. Provide an algorithm for getting a database connection, executing a query, and processing results. How to download the database driver and what it does it represents. What types of database drivers exist in JDBC?
Do I need to register the DB driver? If so, how do I do it?
3. How do I close **the Connection correctly**?

Laboratory work 14. Network programs. Socket connections over TCP / IP. Multithreading. Tasks

Option A

Create a client/server application based on sockets:

1. The client sends a message through the server to another client selected from the list of available clients.
the list.
2. Chat. The client sends a message through the server that all clients receive. The list of clients is stored on the server in a file.
3. When accessing the server, the client receives a randomly selected Maxim Bogdanovich poem from the file.
4. The server sends messages to selected clients from the list. The list of clients is stored in a file

Option B

1. Online game in "Sea battle".
2. Play on the Go network. Tic-tac-toe on a dimensionless (large) field.
To win, you need to line up five in one row.
3. Write a program that scans the network in the specified range
IPaddressed the presence of active computers.
4. Proxy. The program should accept messages from any client,
running on **the TCP protocol**, and send them to the appropriate server.
the server. Change the message during transmission

Please answer the following questions:

1. Seven levels **of the ISO/OSI model**. What was this model designed for?
Assignments for each of the layers and the main protocols that work on them these levels.
2. What is a **MAC**address? Who assigns **MAC**addresses? What network topologies and methods of data transmission are known in these networks?
3. Principles of addressing at the network level? What is an **IP**address, and what classes **of IP**addresses are known? What is a subnet mask and what is it used for?
What reserved **IP**addresses do you know and what do they serve for?
Why **is the MAC**address of a network device not sufficient for network addressing? What are **the ARP** and **RARP** protocols for? How it works
choosing a route for data transmission?

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	---	--	------------------

Laboratory work 15. Java API for XML Processing. Serialization and deserialization of the DOM. Creating an XML document. Tasks

Option A

1. Create an XML file and its corresponding XSD schema.
2. Use simple and complex types, enums, templates, and limit values when developing XSD.
3. Generate a class that matches this description.
4. Create an application for parsing an XML document and initializing a collection of objects with information from an XML file. For parsing, use SAX, DOM, and StAX parsers. Use the Comparator interface to sort objects **Comparator**.
5. Check the XML document using XSD.
6. Define the method that converts the developed XML document to the document specified in each task.

1. The greenhouse.

Plants kept in a greenhouse have the following characteristics:

- Name — name of the plant.
 - Soil — the soil for planting, which can be of the following types: podzolic, ground, sod-podzolic.
 - Origin — place of origin of the plant.
 - Visual parameters — there must be several) - external parameters: stem color, leaf color, average plant size.
 - Growing tips (there must be several) — preferred growing conditions: temperature (in degrees), lighting (light-loving or not), watering (ml per week).
 - Multiplying — propagation: by leaves, cuttings, or seeds. Name the root element Flower.
- Use XSL to convert an XML file to HTML format, where you can display plants by their preferred temperature (in ascending order).

2. The Diamond Fund.

Precious and semi-precious stones contained in the pavilion have the following characteristics:

- The following characteristics: — Name — name of the stone.
- Preciousness — can be precious or semi-precious —
 - Origin — place of extraction.
 - Visual parameters (there must be several) - can be: color (green, red, yellow, etc.), transparency (measured as a percentage of 0-100%), cutting methods (number of faces 4-15).
 - Value — the weight of the stone (measured in carats). Name the root element Gem.
- Use XSL to convert an XML file to XML format, where the root element is the place of origin.

3. Tariffs of mobile companies.

Tariffs of mobile companies can have the following structure:

- Name — the name of the tariff.
 - Operator name — name of the mobile operator that the tariff belongs to.
 - Payroll — monthly subscription fee (0-n rubles).
 - Call prices (there must be several) — prices for calls: inside the network (0-n rubles per minute), outside the network (0-n rubles per minute), to landlines (0-n rubles per minute).
 - SMS price — the price per SMS (0-n rubles).
 - Parameters (there must be several) — availability of your favorite phone number (0-n), billing (12-second, per-minute), connection fee (0-n rubles). Name the root element Tariff.
- Use XSL to convert an XML file to htmlformat, and sort the tariffs by subscription fee in the output.

4. Medicinal products.

Medicinal products have the following characteristics:

	L. N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	---	-------------------------

- Name — name of the drug.
 - Pharm — manufacturing company.
 - Group — a group of drugs that the medicine belongs to (antibiotics, painkillers, vitamins, etc.).
 - Analogs — can be several) - contains the name of the analog.
 - Versions — variants of execution (consistency/type: tablets, capsules, powder, drops, etc.).
- For each version, there can be several manufacturers of medicines with the following characteristics:
- Certificate — certificate of registration of the drug (number, date of issue/expiration, registration organization).
- Package — package (type of package, quantity per package, price per package).
- Dosage — dosage of the drug, frequency of administration; Name the root element Medicine.
- Use XSL to convert an XML file to htmlformat, and sort medicines by price in the output.

Please answer the following questions:

1. What xml parsers exist in Java? Their classification. What is DOM model?
2. How the SAX parser works. Example of getting an object from this parser?
How do I create an event handler for this parser and register it?
3. Why does the SAX parser need an error handler?

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

4. Tasks for independent work of the student and methodological guidelines recommendations for their implementation

SRO volume: 105 hours

Form of control: summary, abstract and presentation.

Time and place of SRO delivery: according to the schedule

Methodological recommendations for the implementation of SRO.

1. Complete a self-training task
2. Read the contents of the suggested information sources for self-training on the issue under consideration.
3. Find additional information yourself.
4. Prepare a summary of the answers to the questions for self-monitoring.

Questions submitted for independent study by students.

Module 1

- 1.1 Implementation of an arithmetic calculator. Testing [1,2, 3, 4, 6, 7, 8]
- 1.2 Implementation of the graphical calculator. Interface [3, 5, 9, 10, 11]
- 1.3 Implementation of the graphical calculator. Logic programming and testing [1,2, 3, 4, 6, 7, 8]
- 1.4 Implementation of a graphical application for testing knowledge. Classes [3, 5, 9, 10, 11]
- 1.5 Implementation of a graphical application for testing knowledge. Graphical interface [1,2, 3, 4, 6, 7, 8]
- 1.6 Implementation of a graphical application for testing knowledge: logic programming [3, 5, 9, 10, 11]
- 1.7 Implementation of a graphical application for testing knowledge: testing [1,2, 3, 4, 6, 7, 8]

Module 2

- 2.1 Implementation of a program for calculating the frequency of occurrence of words in a file. Developing classes [3, 5, 9, 10, 11]
- 2.2 Implementation of a program for calculating the frequency of occurrence of words in a file. Programming logic [1,2, 3, 4, 6, 7, 8]
- 2.3 Implementation of a program for calculating the frequency of occurrence of words in a file. Testing the app [3, 5, 9, 10, 11]
- 2.4 Implementation of a graphical application using a database (using a specific example). Creating classes [1,2, 3, 4, 6, 7, 8]
- 2.5 Implementation of a graphical application using a database (using a specific example). Interface [3, 5, 9, 10, 11]
- 2.6 Implementation of a graphical application using a database (using a specific example). Programming logic. Programming the database [1,2, 3, 4, 6, 7, 8]
- 2.7 Implementation of a graphical application using a database (using a specific example). Connecting the database [3, 5, 9, 10, 11]
- 2.8 Implementation of a graphical application using a database (using a specific example). Testing the app [1,2, 3, 4, 6, 7, 8]

List of literature recommended for performing SRO tasks:

1. Bloch, J. Effective Java / J. Bloch. — 3rd ed. — Boston : Addison-Wesley, 2018. — 412 p.
2. Shildt, G. Java. Complete guide. Java SE 11 / G. Shildt. — 11th ed. - St. Petersburg: Dialektika Publ., 2020. - 1488 p.

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	---	---	-------------------------

3. Fowler, M. Refactoring. Improving the existing code / M. Fowler; translated from English — 2nd ed. - St. Petersburg: Symbol-Plus, 2019. - 464 p.
4. Gradi Butch. Object-oriented analysis and design with examples of applications in C++, Java, and C# / G. Butch; translated from English — 3rd ed. - St. Petersburg: Binom. Laboratory of Knowledge, 2018, 728 p. (in Russian)
5. Ekkel, B. Philosophy of Java / B. Ekkel; translated from English — 4th ed., ispr. - St. Petersburg: Peter, 2018. - 1168 p.
6. Robson, E. Learning Java / E. Robson, E. Freeman; translated from English-St. Petersburg: Peter, 2018. - 720 p.
7. Lyakhovich D. V., Belyakov I. V. Programming on Java / D. V. Lyakhovich, Moscow: Lan, 2022. - 320 p.
8. Object-oriented programming: a textbook for bachelor's and master's degrees / N. M. Samylkina [et al.]; edited by N. M. Samylkina. - Moscow: Yurayt Publishing House, 2022. - 398 p.
9. Horstmann, K. S. Java. Professional library. Volume 1. Fundamentals / K. S. Horstmann; translated from English — 11th ed. - Moscow: Williams, 2019. - 864 p.
10. Horstmann, K. S. Java. Professional library. Volume 2. Extended programming tools / K. S. Horstmann; translated from English — 11th ed. - Moscow: Williams, 2019. - 912 p.
11. Herbert Schildt. A beginner's guide. Java / G. Schildt. - 7th ed. - Moscow: Eksmo, 2019. - 704 p.

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

5. Materials for current, mid-term control, and intermediate certification

Questions for the first border control

1. Defining object-oriented programming and its benefits
2. History of the Java programming language development
3. Basic principles of object-oriented programming
4. The concept of class and object in Java
5. Difference between class instance variables and static variables
6. Assigning and using constructors
7. Encapsulation and how to implement it in Java
8. Example of method overloading in Java
9. Access modifiers and their types
10. Inheritance as a code reuse mechanism
11. Polymorphism and its application
12. Difference between an abstract class and an interface
13. Example of using the interface in Java
14. Purpose of the finalize() method
15. Exception handling mechanisms in Java
16. Structure of the try-catch-finally block
17. Difference between the throw and throws statements
18. Comparison of for, while, and do-while loops
19. Creating and using arrays in Java
20. Passing parameters to Java methods

Questions for the second border control

1. Internal and nested classes in Java
2. Features of anonymous classes
3. Enums and their application
4. The concept of collections and the Collection interface
5. Comparing ArrayList and LinkedList
6. Map interface and its implementations
7. Working with iterators in Java
8. Generics and their syntax
9. Inheritance using generics
10. Event handling in Java
11. Basics of Multithreading
12. The Thread class and its application
13. Difference between the Runnable interface and the Thread class
14. Assigning synchronization in a multithreaded environment
15. Using synchronized blocks
16. Working with files in Java
17. Object serialization
18. Using the Java API
19. Connecting external libraries to a Java project
20. Implementing modularity in Java 9 and later

Questions for final control

1. Object-oriented programming: the essence and principles.
2. A brief history of Java and its features.

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	---	---	-------------------------

3. JVM: purpose, structure, and operation.
4. The difference between the JDK, JRE, and JVM.
5. Structure of the Java program.
6. Classes and objects: basic concepts.
7. Access modifiers.
8. Method overloading.
9. Overloading and overriding: comparison.
10. Inheritance: syntax and usage.
11. Polymorphism: implementation via redefinition.
12. Abstract classes and interfaces.
13. Interfaces with default and static methods.
14. Exceptions: hierarchy, checked/unchecked.
15. Arrays: one-dimensional and multidimensional.
16. Collections: basic interfaces and classes.
17. Lists, sets, and maps: differences.
18. Generics: syntax and purpose.
19. Nested and internal classes.
20. Anonymous classes and lambda expressions.
21. Working with threads: Thread, Runnable.
22. Syncing and managing competitive access.
23. Working with files: File, FileInputStream, BufferedReader.
24. Serialization and deserialization of objects.
25. Working with the Java API.
26. Strings: String, StringBuilder, StringBuffer.
27. Regular expressions in Java.
28. Packages and importing classes.
29. Creating modules in Java 9+.
30. JavaFX and GUI basics.
31. JDBC Basics: Connecting to a database.
32. Working with SQL queries via Java.
33. Network programming and sockets.
34. Working with XML in Java: DOM, SAX.
35. JavaDoc and documentation of code.
36. Logging in Java: Log4J and java.util.logging.
37. Testing Java applications: JUnit.
38. SOLID principles in the Java context.
39. Applying design patterns (for example, Singleton, Factory).
40. Building the Java application architecture.